World Model 与具身智能

从强化学习、世界模型到具身智能体与自主实验室

系统学习教程 | 原理解析 · 代表论文 · 技术脉络 · 动手实践 · 研究问题

基于 liyang.page/blog/ 中的两篇教程重组与讲解：World Model：从强化学习到具身智能的完整入门；面向自主实验室的具身智能体 Tutorial。

本版将聊天中的详细学习内容编排为连续阅读的
PDF。为便于理解，补充了概念对比、实例、研究路线与最小实践代码；部分示意公式与代码为教学改写，并非论文原文逐字摘录。

阅读建议

先读第 1 – 6 章理解世界模型及其技术路线，再读第 7 – 11 章理解自主实验室系统，最后按第 12 – 14 章运行代码、阅读论文并制定研究计划。

资料主页：https://liyang.page/blog/

World Model：https://liyang.page/wm-tutorial/

自主实验室：https://liyang.page/eai-sdl/

目录

第一章 总体知识框架：World Model / Policy / VLA / WAM / Agent

第二章 世界模型基础：状态、动作、预测与规划

第三章 主要技术路线：像素、潜在状态、结构化状态、3D 与仿真

第四章 代表论文：World Models、Dreamer、MuZero、JEPA、Genie、Cosmos

第五章 世界模型与 VLA、WAM、数据和机器人控制

第六章 世界模型的评测与关键难题

第七章 自主实验室：概念与系统架构

第八章 感知与操作：Perception、Manipulation

第九章 具身智能体：规划、记忆、失败恢复与自我改进

第十章 多智能体协作与实验设备调度

第十一章 自主实验室案例：Mobile Robotic Chemist、ORGANA 等

第十二章 动手实验：最小 World Model、DreamerV3 与 TD-MPC2

第十三章 前沿研究问题与两篇教程的知识连接

第十四章 阅读路线、论文清单与最终总结

第一章 总体知识框架

这两篇教程能连成一条主线：先研究 AI 如何预测动作会导致什么结果，再研究如何把预测、感知、行动、规划、记忆与协作组合为现实世界中的智能系统。前一篇重在算法和学习方法，后一篇重在真实科学实验的系统闭环。

概念	核心问题	典型形式
World Model	执行动作后，环境会如何变化?	状态 + 动作 → 未来状态
Policy	当前应该采取什么动作?	状态 → 动作
VLA	如何根据图像和语言指令控制机器人?	图像 + 指令 → 动作
WAM	如何联合预测动作和未来状态?	观测 + 目标 → 动作与未来
Embodied Agent	如何长期完成任务并处理失败?	目标 + 反馈 + 工具 → 计划与执行

例如机器人把一杯水倒入另一个杯子：VLA 决定怎样抓杯与倾斜；世界模型预测倾斜后液体的变化；Agent 将抓取、移动、倾倒和放回组织成子任务，并根据传感器反馈纠错。进入自主实验室后，系统还必须设计实验、使用设备、读

取测量结果并安排下一轮实验。

第二章 世界模型基础

2.1 世界模型的定义

世界模型是一类用于学习环境动态规律的模型。它接收当前状态和动作，预测下一时刻状态，或者对未来状态的概率分布建模。

$$s_{t+1} = f_\theta(s_t, a_t) \quad 或 \quad p_\theta(s_{t+1} \mid s_t, a_t)$$

s_t 是当前环境状态，例如物体位置、关节角和速度；a_t 是行动；s_{t+1} 是预测状态。对于非完全可观测环境，模型还需要从历史观测和动作推断隐藏状态。

2.2 为什么世界模型重要

传统 Model-Free 强化学习依赖真实交互获取大量经验。机器人真实交互成本高，且存在磨损和安全风险。Model-Based RL 则先学习一个动力学模型，再利用模型对多种动作后果进行预测、规划，甚至在模型内部生成模拟轨迹来训练策略。

预测：给定状态与动作，估计未来可能出现什么。

规划：比较多条候选动作序列，选择收益最高的序列。

想象训练：基于学习到的模型展开虚拟轨迹，利用模拟经验更新策略。

注意：模型能够生成逼真的视频，并不意味着能够支持控制。机器人最需要的是动作条件下的反事实预测，即“如果换一个动作，会发生什么”。

2.3 视频生成和世界模型的区别

维度	视频生成模型	控制型世界模型
输入	文本、历史帧、视频	环境状态与候选动作
输出	视觉上合理的后续画面	动作条件下的状态变化
评测	视觉质量、时序一致性等	预测误差、规划有效性、控制成功率等
局限	生成质量不等于物理准确	准确预测也可能不足以覆盖复杂现实

二者并非互斥。某些视频世界模型同样能够输入动作，关键在于模型是否可以对不同动作给出可用的、稳定的后果预测。

第三章 世界模型的主要技术路线

路线	思路	优势与困难
像素级生成	直接预测未来图像或视频	直观，但推理与训练成本高
潜在动态模型	压缩观测后预测隐藏状态	有利于快速想象和策略学习
结构化状态	预测坐标、速度、关节角等变量	适合控制，依赖高质量状态估计
三维世界建模	学习持续、可导航的 3D 场景	适合空间推理，但不等于物理动力学
物理仿真	使用物理引擎模拟状态变化	可控但面临参数与现实差距

这五类并不互斥。实际系统往往将物理仿真、视频模型、状态估计和高频控制器结合为多层架构。

第四章 代表论文与技术演进

4.1 World Models（2018）：在模型内部学习控制

Ha 与 Schmidhuber 的 World Models 把智能体划分成 V、M、C：V（Vision Model）把图像压缩成潜在变量 z；M（Memory/Dynamics Model）预测隐状态随动作的变化；C（Controller）利用隐状态选择动作。核心价值在于让控制器有机会利用已学习的环境模型进行训练。

$$V \rightarrow M \rightarrow C$$

局限：如果预测模型误差持续累积，Agent 可能学会利用模拟器漏洞，而非真正掌握现实环境。

4.2 PlaNet、Dreamer 与 DreamerV3：潜在空间想象

Dreamer 不直接在每一步重建复杂的像素未来，而是在潜在空间中学习环境变化。循环状态空间模型 RSSM 将历史确定性信息与随机潜在信息结合，既能够结合当前观测推断状态，也能够在未来观测缺失时继续展开轨迹。

$$h(t) = f(h(t \quad 1), z(t \quad 1), a(t \quad 1))$$

$$z(t) \qquad p(z(t) \mid h(t))$$

实际训练会同时使用包含当前观测的后验推断，以及只依赖历史的先验预测。训练好世界模型后，Actor 在模型生成的想象轨迹上产生动作，Critic 估计价值，系统由此训练策略。

- Replay Buffer 保存真实交互数据。
- Encoder 和 RSSM 学习状态表示及其动态演化。
- Imagination Rollout 在潜在空间中生成未来状态序列。
- Actor-Critic 在想象轨迹上学习动作和价值。

值得反复思考：为什么预测损失下降不一定带来策略性能提高？因为模型可能学习了对视觉重建有用、却对决策无关的细节。

4.3 MuZero：只预测与决策相关的信息

MuZero 无需完整重建环境图像，而是学习供搜索使用的状态表示、奖励、价值和动作先验。其核心结构可理解为 Representation → Dynamics → Prediction，然后用搜索比较候选决策。它说明世界模型的价值不必通过像素还原来衡量。

4.4 JEPA：预测抽象表征

JEPA 类方法通过预测目标表征而不是逐像素重建图像，尝试聚焦更具语义和任务相关性的变化。然而，不做像素生成并不自动意味着更好的物理理解；仍需在行动预测和下游任务中验证。

4.5 Genie 与 Cosmos：可交互生成世界

Genie 系列探索从视频中学习可交互环境，潜在动作表示尤其重要：普通视频可能没有机器人关节命令，模型可以通过画面变化推断隐式可控因素。Cosmos 代表将物理世界生成、预测、机器人训练数据与评测组成基础平台的方向。

归纳：World Models → Dreamer 偏向模型学习与控制；MuZero 偏向搜索与决策；JEPA 偏向表征预测；Genie、Cosmos 偏向大规模世界生成与交互。它们不是单条简单的技术继承链。

第五章 World Model、VLA、WAM 与数据

5.1 VLA：从视觉和语言到行动

$$a(t) = \pi_\theta(o(t), \text{instruction})$$

VLA（Vision-Language-Action）将摄像头观测和语言任务映射为机械臂或机器人动作。代表工作包括 RT 系列、OpenVLA 与 π 系列。它主要回答“我现在该怎么做”。

5.2 World Model：从行动到后果

$$o(t+1) = f_\theta(o(t), a(t))$$

世界模型根据状态和动作预测后果。对于抓取苹果，它关心动作后苹果是否被抬起、是否滑落，以及下一步会不会碰撞。

5.3 WAM：联合建模未来动作与未来状态

$$p_\theta(a(t{:}t+H), o(t+1{:}t+H) \mid o(\leqslant t), \text{instruction})$$

World Action Model 试图在同一框架中联系动作生成和未来状态预测。DreamZero 是代表性方向之一。联合建模可能增加训练信号，但计算需求、数据对齐和控制实时性也是挑战。

5.4 数据需求有何不同

任务	典型数据	难点
VLA	观测、语言、专家动作	真实示范数据昂贵
World Model	状态、动作、下一状态	需要丰富的成功与失败转移
WAM	视频、语言、动作与未来观测	时空与动作精确对齐
视频生成	文本和连续图像序列	物理可控性未必充分

世界模型需要学习动作后果，因此失败数据同样重要。相反，动作模仿往往更强调高质量成功示范。两者的训练数据设计不应混为一谈。

5.5 机器人中的分层控制

层次	主要任务	典型模块
高层	目标解释、长程规划	LLM、Agent、长程世界模型
中层	预测后果、选动作序列	Action-conditioned WM、MPC
低层	快速稳定执行	快速 Policy、反馈控制器
安全层	碰撞检测、异常处理	约束、监控和人工接管

第六章 世界模型如何评测?

不能只评价生成画面的主观质量。至少应区分预测本身、决策价值和真实系统可靠性三类指标。

单步与多步状态预测误差：预测步数变大时误差怎样变化。

动作条件一致性：不同动作是否导致有区分度且符合环境的未来。

奖励与事件预测：接触、碰撞、失败、任务完成等是否被预测。

闭环控制表现：任务成功率、完成时间、累计奖励、恢复成功率。

可靠性：置信度校准、分布外泛化、异常检测与人类接管频率。

特别要关注现实中的偏差：一个在简单物理环境中学得很好的世界模型，可能因为液体、形变、遮挡、摩擦和接触条件变化而失效。

第七章 自主实验室基础

Self-Driving Laboratory（SDL，自主实验室）尝试把科学目标、实验设计、机器人执行、仪器测量与结果分析连接成闭环。它的重点不仅是机器代替人做重复劳动，而是系统能够利用实验结果决定下一轮条件。

科学目标 → 实验设计 → 机器人执行 → 数据采集 → 结果分析 → 下一轮实验

7.1 自动化与自主的区别

自动化：人预先固定流程，例如把 10 微升试剂从一个容器移到另一个容器。自主：系统能够根据实验结果选择新的实验条件、调整计划并处理部分执行异常。自主不意味着完全无人监督，高风险实验必须仍有安全审查与授权机制。

7.2 四层系统结构

模块	核心能力	实例
Perception	物体、空间和实验状态感知	识别透明试管、测液面
Manipulation	机器人动作和物理交互	抓取、移液、旋盖
Embodied Agent	规划、记忆、验证和恢复	失败后修复执行方案
Multi-Agent	协作与资源调度	多机械臂、仪器和机器人协调

第八章 Perception 与 Manipulation

8.1 为什么实验室感知不等于普通目标检测

识别图像中有烧杯只是第一步。实验室还要求判断烧杯里的液体是否混匀、是否有沉淀、温度或反应状态是否达标。透明、反光、密集遮挡、微小器材和液体都会使普通视觉模型遇到困难。

- Object Perception：识别试管、枪头、容器和仪器。
- Spatial Intelligence：估计三维几何、位姿、可达性和遮挡关系。
- State Perception：判断混合、温度、反应或实验进度等状态。

仪器读数、压力、温度、力觉与视觉经常需要融合，不能把实验状态完全等同于图像分类。

8.2 操作层：VLA、世界模型和 WAM 如何配合

VLA 根据视觉和语言指令产生动作；世界模型预测动作后果；WAM 将未来动作与状态预测纳入更统一的框架。现代具身系统往往需要分层控制，避免把所有实时任务压给一个大型生成模型。

8.3 π0.7 与 DreamZero

第二篇教程通过 π0.7 和 DreamZero 说明机器人策略正在融入记忆和预测能力。π 系列关注跨任务、跨环境的动作策略；DreamZero 关注动作与未来视频共同建模。不能仅由论文中报告的实验指标推断它们在所有机器人平台上都有同样效果。

第九章 Embodied Agent：规划、记忆和自我改进

长程任务由多个步骤组成，即使单步成功率很高，任务整体也容易失败。假设 40 步相互独立，每步成功率为 95%，则总体成功率约为 0.95^40 ≈ 12.85%。这是说明误差累积的教学示例，不是现实系统的一般概率公式。

9.1 Agent 在系统中的职责

任务分解：把总体目标变成可执行子任务。

状态维护：知道哪些步骤完成，哪些待执行。

验证与纠错：用观测检查真实结果是否符合预期。

恢复与重新规划：发生失败时回退、调整或请求介入。

技能记忆：复用历史成功和失败经验。

9.2 ASPIRE：失败后如何改进

ASPIRE 代表从失败中诊断、修复和积累技能的路线。教程强调闭环执行引擎、技能库和进化搜索：执行过程中记录反馈，诊断失败原因，调整操作程序，然后保存经验证有用的技能。

执行 → 反馈 → 诊断 → 修复 → 技能积累 → 再执行

记忆与自我改进必须区分：Memory 是保存与检索历史；Self-Improvement 要求这些历史被验证并真正改变未来决策。仅保存一条失败日志不等于系统有了自进化能力。

9.3 Agent Memory 与 World Model 的互补

世界模型学习状态转移规律，记忆系统保存历史任务、操作经验和技能。将两者结合，Agent 可先检索类似任务的经验，再用世界模型验证候选动作的潜在结果，最后执行并更新经验。记忆可能过时、世界模型可能错误，两者都需要可靠性评估。

第十章 Multi-Agent 与动态协作

真实实验室可能有机械臂、移动机器人、离心机、检测仪器及人类研究者。系统不仅要规划动作，还要决定谁执行、何时执行、设备如何共享以及如何处理失败交接。

10.1 RoCo：语言协商与物理执行

RoCo 将不同机器人的状态和能力交由智能体进行协商，再把协作方案交给物理规划器。语言协商可以促进任务分工，但并不能替代运动学、碰撞检测或可达性验证。

10.2 HOLA：开放环境中的队友变化

HOLA 关注未知队友和变化团队规模下的协作适应。对实验室而言，系统需要在新机器人加入、设备更换、任务负载变化后继续稳定运行，而不是每次都重新设计完整流程。

第十一章 自主实验室代表案例

11.1 Mobile Robotic Chemist (2020)

该工作展示移动机器人连接科学设备与贝叶斯优化，依据实验结果选择后续条件。教程介绍其在约 8 天内完成 688 次实验的案例。其意义是形成真实物理实验闭环；局限是操作与设备环境仍高度结构化。

11.2 ORGANA (2025)

ORGANA 强调从自然语言科学指令到机器人实验执行的转换，使用户能够以更高层次的任务语言描述实验。主要系统链路是自然语言目标 → 任务解析 → 操作规划 → 实际执行 → 结果分析。

11.3 Embodied Science Agent (2026)

教程介绍统一科学规划与多设备控制接口的探索，并通过实验条件优化案例展示科学智能体如何与真实物理执行结合。理解此类系统时应区分科学优化算法、机器人动作控制和实验测量三个不同环节。

第十二章 动手实验与复现路线

12.1 从一个二维世界模型开始

先实现状态转移预测，再增加多步规划。以下代码是完整的 PyTorch
教学示例，学习状态、动作到下一个状态的映射；与 Dreamer
等正式算法不同，它没有视觉编码器、奖励模型或潜在记忆机制。

```
import torch
from torch import nn

torch.manual_seed(42)
device = "cuda" if torch.cuda.is_available() else "cpu"

N = 10000
states = torch.rand(N, 2) * 10
actions = torch.randn(N, 2) * 0.3
noise = torch.randn(N, 2) * 0.02
next_states = states + actions + noise

inputs = torch.cat([states, actions], dim=-1).to(device)
targets = (next_states - states).to(device)

model = nn.Sequential(
    nn.Linear(4, 128), nn.ReLU(),
    nn.Linear(128, 128), nn.ReLU(),
    nn.Linear(128, 2)
).to(device)

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

for epoch in range(300):
    prediction = model(inputs)
    loss = loss_fn(prediction, targets)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 50 == 0:
        print(epoch + 1, loss.item())
```

```
model.eval()
state = torch.tensor([[2.0, 3.0]], device=device)
action = torch.tensor([[0.5, -0.2]], device=device)
with torch.no_grad():
    delta = model(torch.cat([state, action], dim=-1))
    predicted_next = state + delta
print(predicted_next.cpu())
```

练习一：评估训练集与测试集误差。练习二：把一步预测迭代十次，比较预测漂移。练习三：加入墙壁碰撞或摩擦等非线性规则。练习四：枚举动作序列并选择距离目标最近的一条。

12.2 Model Predictive Control 的直觉

假设机器人目标位置为 g，可以先利用世界模型模拟多组动作序列，选择预计到达位置最接近 g、动作幅度又不过大的方案。实际系统只执行序列开头的一部分，再利用新观测重新规划。

$$a^{*} = \arg\min \left\{ \left\| s_{(t+H)} - g \right\|^{2} + \lambda \sum \left\| a(k) \right\|^{2} \right\}$$

12.3 DreamerV3 安装与运行示例

```
git clone https://github.com/danijar/dreamerv3.git
cd dreamerv3
python -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
pip install -r requirements.txt

python dreamerv3/main.py \
  --logdir ~/logdir/dreamer/debug \
  --configs crafter debug \
  --run.train_ratio 32
```

这组命令来自教程示例，不代表已在本机验证。实际复现时需要根据目标版本确认 JAX、CUDA、依赖文件和参数名称。阅读源码重点查 Encoder、RSSM、Reward Predictor、Actor、Critic 和 Replay Buffer。

12.4 TD-MPC2 安装与运行示例

```
git clone https://github.com/nicklashansen/tdmpc2.git
cd tdmpc2
conda env create -f docker/environment.yaml
conda activate tdmpc2
python train.py task=walker-walk steps=700000

# ■■■■■■■■■■■■■■■■■■■
python train.py task=walker-walk obs=rgb
```

关注结构化状态观测与 RGB 视觉观测的差异。在图像输入下，模型除了学习动力学，还要解决视觉表征、部分可观测性和感知噪声问题。

12.5 复现实验应该如何记录

环境：OS、Python、PyTorch/JAX、CUDA 版本与显卡型号。

数据：采样方式、总步数、任务种子和数据拆分。

模型：架构、参数量、时间预测长度和训练超参数。

指标：累计奖励、成功率、单步/多步误差、训练耗时。

对照：去掉世界模型、改变预测步数、调整策略更新频率等消融。

第十三章 前沿研究问题与知识连接

问题	形成原因	研究切入点
长程误差累积	一步误差在多次展开中放大	分层预测、不确定性感知
物理可靠性	生成模型可能违背接触规律	物理约束与错误识别
真实数据稀缺	机器人实验昂贵	视频辅助学习、仿真迁移
动作与预测协同	生成视频与控制目标不同	WAM、多任务损失设计
长程执行失败	步骤多、异常多	验证、恢复和重规划
技能知识无法复用	经验碎片化	Agent Memory、Skill Library
自我改进缺证据	日志增加不等于能力提高	经验验证与持续评测
开放式协作	设备和队友变化	动态规划、工具接口标准化

跨领域研究的一个自然组合是 Memory → Planning → World Prediction → Action → Feedback → Memory Update。研究重点应落在可衡量的收益上：成功率有没有提升、真实试错是否减少、恢复能力是否增强，而不是仅仅将几个模块串接。

第十四章 学习路线与结论

阶段	内容	阶段成果
1. 基础（3 5 天）	MDP、模型化强化学习、状态转移	能区分 Policy 和 WM
2. 经典论文（5 7 天）	World Models、RSSM、DreamerV3	可画出训练闭环
3. 控制规划（5 7 天）	MuZero、TD-MPC2、MPC	能解释候选动作如何比较
4. 具身学习（5 7 天）	OpenVLA、DreamZero、WAM	能区分 VLA、WM、WAM
5. 系统整合（5 7 天）	ASPIRE、ORGANA、RoCo	能设计模拟 SDL 闭环

建议优先阅读的论文与项目

World Models（2018）：V M C 架构与在梦境中学习。

DreamerV3：RSSM、Imagination、Actor-Critic。

TD-MPC2：潜在动态、连续控制与 MPC。

MuZero：面向决策的环境建模和树搜索。

OpenVLA：视觉语言动作策略与机器人数据。

DreamZero：视频与动作的联合建模。

ASPIRE：失败恢复、技能记忆和自我改进。

ORGANA / RoCo / HOLA：实验规划与多机器人协作。

六点最终认识

世界模型要能预测行动后果，而不只是生成逼真的画面。

用于控制的模型需要保存决策相关的信息，而不一定完整重建环境。

VLA 负责动作，WM 负责后果；WAM 尝试在统一框架内连接两者。

长期任务可靠性依赖规划、反馈、验证、记忆和失败恢复。

自主实验室的价值在于科学设计、物理实验和结果反馈构成真实闭环。

具身智能是由感知、操作、预测、规划、协作和安全机制组成的系统。

原始教程及参考项目

https://liyang.page/blog/

https://liyang.page/wm-tutorial/

https://liyang.page/eai-sdl/

https://worldmodels.github.io/

https://github.com/danijar/dreamerv3

https://github.com/nicklashansen/tdmpc2

https://github.com/danijar/daydreamer

https://github.com/openvla/openvla

说明：本 PDF 是为学习方便编排的中文教学整理，不替代原教程或原论文。部分公式为简化记法；具体方法、实验数据和实现配置，请以原论文与对应仓库版本为准。